

Playwright-Notes-1

- Its an open source test automation library
- Allows you to control a web browser automatically

Why would anyone use this?

-  **Testing** — automatically check if your website works correctly
-  **Scraping** — collect data from websites
-  **Automation** — do boring repetitive browser tasks automatically

- Simple analogy/example:

Imagine you have a **toy robot**  that can control a web browser for you.

You can tell the robot:

- "Open Google"
- "Type 'cats' in the search box"
- "Click the search button"
- "Tell me what you see on the screen"

And the robot just **does it automatically** without you touching the keyboard or mouse.

That's Playwright!

- It was created specifically to accommodate the needs of end to end testing
- Supports all engines and browsers including Chromium, Firefox,Webkit

Advantages:

- Multibrowser support
- Easy setup and configuration
- Supports multiple languages: JS, Typescript,Python,etc
- CI/CD integration support
- Allows Parallel browser testing

Javascript Basics for Automation

- Javascript is used to write logic to automate tests/actions
- Cypress and Playwright mainly use JS to execute tests

- **Variables in JS**-used to store values-help make code dynamic instead of hardcoding data-allows reusability to perform various actions/functions
- let and const are used to initialise,declare and define variables-these keywords are used to create variables
- Use const when variable value doesn't change-fixed value
- Use let when variable value changes
- Data types (type of value stored in JS) include: Strings,Numbers,Boolean

Other data types include:

- Array-stores multiple values in list format Eg: `let hobbies=["Painting","Reading", "Travelling"]`
-
- Object-stores related data in key-value pairs Eg:for storing user info on a website, json files,etc -used in JS automation tools like Cypress and Playwright

Eg: `let userdetails={`

`Username:"rux",`

`password:"12345"`

`}`

- **Functions**-help group multiple steps of a task into a reusable code block-helps in avoiding writing the same code and provides more cleaner, readable and maintainable code

Eg: `function introduceyourself(){`

`console.log("Hello, my name is Amrutha and I am 23 years old")`

`}`

`introduceyourself()`-invoke it here

- **Arrow Functions:** Shorter and modern way of writing javascript functions-commonly used in test automation tools

Eg: `const myarrowfunction=()=>{`

`console.log("This is an arrow function")`

`}`

`myarrowfunction()`-invoke it here

Here myarrowfunction is a const variable name

- **Parameters/arguments**-allow functions to accept inputs making them flexible and reusable

```
Eg: function introduceyourself(myname,age){  
    console.log("Hello, my name is "+myname+" and I am "+age+" years  
old")  
}
```

introduceyourself("Amrutha", 23)-invoke it here with parameters

- **Return values** allow the function to send or return values back after execution of the function

Eg: const getAge=()=>{

age=23

return age

}

console.log(getAge())

or assign it to a variable like so: //or

//const ageVal=getAge()

//console.log(ageVal)

- **Operators :Arithmetic**-used to perform mathematical calculations:+,-,*,/,%
- **Comparison**:===,!==,>,<=>,<-used to compare values and returns true or false
- **Logical**: && (AND-both true),||(OR-any one true),!(NOT-reverse result)-helps create complex decision making logic
- **Conditions-(if/else if/else)**-allows the program to make decisions depending on whether a specific condition is satisfied or not-used frequently for validations and result checks

Eg (conditional statements-if/else): let num=24

```
if(num%2==0){
```

console.log(num+" is an even number")

```

}

else{

    console.log(num+" is not an odd number")

}

```

Eg: let a=10

```

let b=20
if(a>b){
    console.log(a+" is greater than "+b)
}
else if (a<b){
    console.log(a+" is less than "+b)
}
else{
    console.log(a+" is equal to "+b)}

```

- **Loops**-repeat the same steps multiple times-useful when handling multiple elements or test data

Eg: for(let i=0;i<11;i++){
 console.log(i)
 }

Eg: let fruits=["Apple", "Banana", "Cherry"]
 for(const fruit of fruits){
 console.log(fruit)
 }

- **Async and Await**
- Some tasks/operations take time to perform like loading data-we use async functions-a function which takes time to complete
- Await functions pause the execution until the task finishes before moving on to the next step-waits until the promise has been finished before executing the next line
- Importance of async and await is that they ensure correct execution of steps in correct order so that it prevents unstable behavior
- This concept is critical for reliable automation scripts
- Eg: async function loadData(){
 await fetch(<https://example.com>)
 }

- **Whenever you use await inside a function it should be declared as a async function**
- **Promises**-an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value.

Think of it like ordering food at a restaurant — you get a "ticket" (the promise) immediately, and later it either gets fulfilled (food arrives) or rejected (they're out of that dish).

The 3 States

A Promise is always in one of these states:

Pending — initial state, still waiting

Fulfilled — operation completed successfully

Rejected — operation failed

Basic Syntax:

```
const myPromise = new Promise<string>((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("It worked!"); // → fulfilled
  } else {
    reject("Something went wrong"); // → rejected
  }
});
```

Consuming a Promise

With .then() / .catch():

```
myPromise
  .then(value => console.log(value)) // "It worked!"
  .catch(err => console.error(err))
  .finally(() => console.log("Done")); // always runs
```

With async/await (cleaner, more readable):

```
async function run() {
  try {
    const result = await myPromise;
    console.log(result); // "It worked!"
  } catch (err) {
    console.error(err);
  }
}
```

Common Real-World Usage

```
// Fetching data from an API
async function getUser(id: number) {
  const response = await fetch(`/api/users/${id}`); // returns a Promise
  const user = await response.json();              // also a Promise
  return user;
}
```

- **Classes**

- A container used to group similar entities/actions together in one place-groups related data and actions together
- Improves code readability, structure and organization
- In automation its used to represent pages or modules-structures automation frameworks

- **Methods**

- Functions inside a class that perform specific functions related to a class

- **Attributes/Properties/Fields**

- Variables inside a class

- **Constructor**

- A method used to initialize values/variables
- Automatically executes when an object is created

- **Objects**

- Instances of a class/instance created from a class (usually outside the class)
- It can access attributes and methods of a class using the **'.'** operator
- They execute the methods of a class

Eg: class Car{

//color="Red" //color and speed are attributes/properties of the class Car
// speed=200

constructor(model){ **//constructor used to initialise values constructor can or cannot have parameters**

 this.color="Red"
 this.speed=200
 this.model=model

}

//constructor(){ //constructor without parameters

```

        //this.speed=200

    // }

    car_color(){
        console.log("The color of the car is "+this.color)
    }
    car_speed(){
        console.log("The speed of the car is "+this.speed)
    } //car_color and car_speed are methods of the class car
}

const mycar=new Car() //mycar is an object of the class Car
mycar.car_color()
mycar.car_speed()
console.log(mycar.color)
console.log(mycar.speed)
const anothercar=new Car("BMW") //anothercar is another object of the class Car
and we are passing the model parameter to the constructor to initialise the
model attribute for anothercar object
console.log(anothercar.model)
console.log(mycar.model) //mycar.model is undefined because we have not
initialised the model attribute for mycar object, we have only initialised it for
anothercar object using the constructor with parameters

```

- **Import/Export**
- Used in multiple files in large projects
- Enhances readability, organization and maintainability in larger projects
- **Syntax: import {<component>} from <./filename (file 2)>**
in file 2, export <componentname> in file 1
- Component can be a class, function, etc.

Typescript Basics for Automation

Note: ‘;’ is optional in typescript but its better to include it

- Typescript is Javascript with types/type safety. This is demonstrated in the below example:

JavaScript lets you do this without complaining:

Eg: let age = 25

```
age = "hello" // no error! JS doesn't care
```

```
age = true // still no error!
```

TypeScript says "wait, what TYPE of thing is this?":

```
let age: number = 25
```

```
age = "hello" // ❌ ERROR! you said it was a number!
```

```
age = true // ❌ ERROR! still a number!
```

```
age = 30 // ✅ fine!
```

TypeScript is like a **strict teacher** who checks your work.

Imagine you have a **box labeled "BALLS ONLY"**

- **JavaScript** lets you put balls, books, bananas, anything in it — no rules
- **TypeScript** sees you putting a banana in and says **"HEY! This is a BALLS ONLY box!"**

Why is that useful?

// TypeScript catches mistakes BEFORE you run the code

```
function add(a: number, b: number) {  
    return a + b  
}
```

```
add(5, 10) // ✅ 15
```

```
add(5, "10") // ❌ caught immediately! before it even runs
```

Without types, `add(5, "10")` in JavaScript silently gives you `"510"` instead of `15` — a bug that could be hard to find!

TypeScript is just JavaScript with a **safety net** — and in the end it all converts back to plain JavaScript anyway since browsers only understand JS.

- Typescript adds safety by catching errors during development instead of runtime.

- Typescript is used for automation as it provides auto-complete, fewer bugs with more clear code-this makes automation frameworks easier to build, read and maintain.
- The core idea is types which determine the type of value that a variable stores-prevents invalid data and reduces errors/mistakes
- To install typescript globally use the command: **npm install -g typescript**
- **Variables**-used to store values that can be reused in the code-in typescript we can define the type of each variable
- Use **const** to declare a variable by default-const variable values cannot be changed. Use **let** only when the value of the variable changes
- We cannot directly execute typescript code so first we have to convert it into javascript code and then only execute.Follow these steps:

Open bash>tsc <typescript [filename.ts](#)>>tsc --ignoreConfig

- The equivalent .js file gets created and this can be executed in bash using **node <javascript [filename.js](#)>**
- We can combine commands using “&&”

Eg: tsc <typescript [filename.ts](#)>&&node <javascript filename.js>

Eg: const myname:string=”Rux”

console.log(myname)

let myage:number=23

myage=24

console.log(myage)

console.log(`My name is \${myname}`) **//placeholder value for back ticks (same with javascript as well)**

(or)

console.log(“My name is “ +myname) **//placeholder value for single and double quotes (same with javascript as well)**

Const variables should always be assigned/initialised with a value immediately when created-we cannot do this:

const a

a=10 //results in error

- With let variables we can create the variable without a value and assign the value to it later like so:

```
let a
```

```
a=10 //won't result in an error
```

- This applies to both javascript and typescript
- **Data types:** number, string, boolean, Arrays-store multiple values of same data type in a list and stores multiple values of different data type in a tuple

```
Eg: let isStudent:boolean=true //boolean
```

```
let myname1:string="Rux" //string
```

```
let myage1:number=23 //number
```

```
let hobbies:string[]=["coding","gaming","traveling"] //Array-list
```

```
console.log(hobbies[2])
```

```
let me:any[]=["Rux",23,true] //Array-tuple
```

```
console.log(me[0])
```

```
let nums:Array<number>=[1,2,3,4,5] //Alternative way to declare an array
```

```
console.log(nums[3])
```

- **Objects**-group related data together with structure and types-includes a key value pair like so:

```
Eg: let person:{name:string,age:number}={name:"Rux",age:23}
```

```
console.log(person.name)
```

```
console.log(person.age)
```

We use interfaces with objects to make test data more organised (discussed later)

- **Functions**-define types for inputs (parameters) and return values-ensures correct data is passed and returned
- **Parameters**-allows functions to receive input values with specific types
- **Return types**- defines what type of value a function should return
- **Arrow Functions**-shorter and modern way/syntax of writing functions-these are more commonly used than normal functions
- **Optional Parameters**-allows functions to accept values that may or may not be provided - adds flexibility to function calls
- Eg: function greeting():void{ //normal function-return type is void

```
console.log("Hi, how are you?")
```

```
//return 2 //This will cause an error because the return type is void
```

```
}
```

```
greeting()
```

```
const farewell=():void=>{ //arrow function-return type is void
```

```
    console.log("Goodbye!")
```

```
}
```

```
farewell()
```

```
const add=(a:number,b:number):number=>{ //arrow function with parameters and return type
```

```
    return a+b
```

```
}
```

```
console.log(add(5,10))
```

```
function printname(name?:string):void{ //function with optional parameter and return type
```

```
if(name){  
    console.log(`My name is ${name}`)  
}  
  
console.log("Name not provided")  
}  
  
printname("Rux")  
  
printname()
```

- **Operators**-Arithmetic,comparison,logical-same as in Javascript

All JavaScript operators work exactly the same in TypeScript. TypeScript just adds type checking on top.

All the usual operators work the same:

// Arithmetic

let a = 10 + 5 // 15

let b = 10 - 5 // 5

let c = 10 * 5 // 50

let d = 10 / 5 // 2

let e = 10 % 3 // 1 (remainder)

// Comparison

5 == "5" // true (loose, doesn't check type)

5 === "5" // false (strict, checks type too)

5 != 3 // true

5 > 3 // true

```
// Logical
```

```
true && false // false
```

```
true || false // true
```

```
!true // false
```

The only difference is TypeScript will warn you if you use operators on wrong types:

```
let num: number = 10
```

```
let str: string = "hello"
```

```
let result = num + str // ⚠️ TypeScript warns you!
```

```
// JS would silently give "10hello"
```

TypeScript also adds one extra useful operator:

```
// ?? (Nullish coalescing) - use default if value is null/undefined
```

```
let name = null ?? "Guest" // "Guest"
```

```
let age = 25 ?? 0 // 25 (already has a value)
```

Though this also exists in modern JavaScript now too.

In short — operators are identical, TypeScript just **stops you from using them on mismatched types** before your code even runs.

- Conditions-allow the program to make decisions based on whether a condition is satisfied or not-same as Javascript

Eg: // exactly the same as javascript

```
let age = 18
```

```
if (age >= 18) {
```

```

        console.log("adult")
    } else if (age >= 13) {
        console.log("teenager")
    } else {
        console.log("kid")
    }

```

The only time TypeScript gets involved is if you use wrong types in the condition:

```
let age: number = 18
```

```

if (age === "18") { // ❌ TypeScript warns you!
    console.log("adult") // comparing number to string
}

```

JavaScript would silently run that without complaining, TypeScript catches it immediately.

- **Loops**-same as in Javascript

```

// for loop
for (let i = 0; i < 5; i++) {
    console.log(i) // 0 1 2 3 4
}

```

```

// while loop
let i = 0
while (i < 5) {
    console.log(i) // 0 1 2 3 4
}

```

```
    i++  
  }
```

```
// do while loop
```

```
let i = 0
```

```
do {
```

```
  console.log(i) // runs at least once even if condition is false
```

```
  i++
```

```
} while (i < 5)
```

```
// for of loop (arrays)
```

```
let fruits = ["apple", "banana", "mango"]
```

```
for (let fruit of fruits) {
```

```
  console.log(fruit) // apple banana mango
```

```
}
```

```
// for in loop (objects)
```

```
let person = { name: "Alice", age: 25 }
```

```
for (let key in person) {
```

```
  console.log(key) // name age
```

```
}
```

The only time TypeScript gets involved is when types are wrong:

```
let numbers: number[] = [1, 2, 3]
```

```

for (let num of numbers) {
    console.log(num + "hello") // ⚠️ TypeScript warns you!
                                // adding number to string
}

```

- **Async and await** handle functions that take time to complete-same as Javascript

Eg: // fetching data - exactly like javascript

```

async function getUser() {
    let response = await fetch("https://api.example.com/user")

```

```

        let data = await response.json()
        console.log(data)
    }

```

The only difference is TypeScript lets you type what the function will return:

```

// javascript

async function getUser() { // no idea what this returns
    let response = await fetch("https://api.example.com/user")
    return await response.json()
}

```

```

// typescript

async function getUser(): Promise<string> { // we know it returns a string
    let response = await fetch("https://api.example.com/user")
    return await response.json()
}

```

`Promise<string>` just means — *"this function is async and will eventually give back a string"*

With try/catch for errors — also the same:

```
async function getUser() {  
  try {  
    let response = await fetch("https://api.example.com/user")  
    let data = await response.json()  
    console.log(data)  
  } catch (error) {  
    console.log("something went wrong!", error)  
  }  
}
```

- **Promises**-In TypeScript, Promises are generic — `Promise<T>` tells you what type the resolved value will be:

```
function fetchName(): Promise<string> {  
  return Promise.resolve("Alice");  
}
```

```
async function fetchAge(): Promise<number> {  
  return 30; // TypeScript auto-wraps in a Promise  
}
```

- **Interfaces**- defines the structure of objects-makes test data clean, reusable and easy to maintain-we use interface when we use objects as it provides a more readable, cleaner way of representing objects

Eg:

```
interface MyBio {  
  
    name: string;  
  
    age: number;  
  
}
```

```
let Amrutha:MyBio={  
  
    name:"Rux",  
  
    age:23  
  
}
```

```
console.log(Amrutha.name)
```

```
console.log(Amrutha.age)
```

- **Type/Type aliases (alternative to interface)**-another way to define a data structure-used for custom types or limited value sets-if we want to restrict the user to enter only a limited number of specific values like “Yes” or “No” then we can use type

```
Eg: type MyInfo={  
  
    food:string;  
  
    drink:string;  
  
}
```

```
let AmruthaInfo:MyInfo={  
  
    food:"Pizza",  
  
    drink:"Coffee"  
  
}
```

```
console.log(AmruthaInfo.food)
```

```
console.log(AmruthaInfo.drink)
```

- Type can be used in cases where the user is only allowed to enter a specific, limited set of values/choices

Eg: `type response="Yes"|"No"|"Maybe" //allows user to enter a limited number of values`

```
let myResponse:response="Yes"
```

```
console.log(myResponse)
```

`//let anotherResponse:response="not sure" //This will cause an error because the value is not one of the allowed values`

- **Classes, methods and objects in Typescript are the same as Javascript**

Eg: JavaScript class:

```
class Person {
  constructor(name, age) {
    this.name = name
    this.age = age
  }

  greet() {
    console.log(`Hi I am ${this.name}`)
  }
}
```

```
let p = new Person("Alice", 25)
```

```
p.greet() // Hi I am Alice
```

TypeScript class — same thing, just with types:

```
class Person {
```

```
name: string    // ← declare attributes with types
```

```
age: number
```

```
constructor(name: string, age: number) { // ← typed parameters
```

```
    this.name = name
```

```
    this.age = age
```

```
}
```

```
greet(): void { // ← void means returns nothing
```

```
    console.log(`Hi I am ${this.name}`)
```

```
}
```

```
}
```

```
let p = new Person("Alice", 25)
```

```
p.greet() // Hi I am Alice
```

TypeScript also adds access modifiers which JavaScript doesn't really have:

```
class Person {
```

```
    public name: string    // anyone can access
```

```
    private age: number    // only accessible inside the class
```

```
    protected id: number  // accessible inside class and subclasses
```

```
    constructor(name: string, age: number, id: number) {
```

```
        this.name = name
```

```
        this.age = age
```

```
        this.id = id
    }
}
```

```
let p = new Person("Alice", 25, 1)
```

```
console.log(p.name) //  works
```

```
console.log(p.age) //  TypeScript error! age is private
```

Inheritance also the same:

```
class Animal {
    name: string

    constructor(name: string) {
        this.name = name
    }

    speak(): void {
        console.log(`${this.name} makes a sound`)
    }
}
```

```
class Dog extends Animal {
    breed: string

    constructor(name: string, breed: string) {
```

```

    super(name)    // same as javascript

    this.breed = breed
}

speak(): void {
    console.log(`${this.name} barks!`)
}
}

```

```

let d = new Dog("Rex", "Labrador")

d.speak() // Rex barks!

```

- **Import and export**-splits code into multiple files and improves code organization and maintainability-works like javascript
- **Typescript is just Javascript with additional type safety-removing types makes it javascript**

